

## 4. break Statement – Stop a Loop Intentionally

---

### Interview Rule

Use break when you want to **terminate the nearest loop or switch statement immediately**.

After break executes, control moves to the first statement **after** the loop.

---

### Syntax

```
break;
```

---

### Real-World Uses

- Searching an element in an array
- Password validation
- Exit menu-driven programs
- Stop processing when an error occurs
- Finding the first matching record

Example:

```
int[] arr = {5, 8, 12, 20};

for (int num : arr) {
    if (num == 12) {
        System.out.println("Found");
        break;
    }
}
```

---

# Hidden Interview Facts

## Q1. Which loop does break terminate?

Only the **nearest (innermost)** loop.

Example:

```
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 2)  
            break;  
  
        System.out.print(i + " " + j + " ");  
    }  
}
```

Output

```
11 21 31
```

The outer loop continues.

---

## Q2. Does break terminate an if statement?

No.

break works only with:

- loops
  - switch
- 

## Q3. Can break be used outside a loop?

No.

```
break;
```

Compiler Error.

---

#### Q4. Does break stop program execution?

No.

It only exits the current loop/switch.

---

## Labeled Break (Advanced Interview)

Java supports **labeled break**.

Example:

```
outer:
for(int i=1;i<=3;i++)
{
    for(int j=1;j<=3;j++)
    {
        if(j==2)
            break outer;

        System.out.print(i+" "+j+" ");
    }
}
```

Output

```
11
```

Entire outer loop stops.

Interviewers love this question because many developers never use labeled breaks.

---

## Tricky Output Questions

### Question 1

```
for(int i=1;i<=5;i++)
{
    if(i==3)
        break;

    System.out.print(i+" ");
}
```

Output

```
1 2
```

---

### Question 2

```
for(int i=1;i<=3;i++)
{
    for(int j=1;j<=3;j++)
    {
        if(j==2)
            break;

        System.out.print(j);
    }
}
```

Output

```
111
```

---

### Question 3

```
int i=1;

while(true)
{
    if(i==4)
        break;

    System.out.print(i++ +" ");
}
```

Output

1 2 3

---

#### Question 4

```
for(int i=0;i<5;i++)
{
    break;
    //System.out.println(i);
}

System.out.println("Done");
```

Output

Done

---

## break vs return (Very Common Interview)

| break             | return              |
|-------------------|---------------------|
| Exits loop/switch | Exits entire method |

|                              |                         |
|------------------------------|-------------------------|
| Program continues after loop | Method ends immediately |
| Used inside loops/switch     | Used inside methods     |

Example

```
for(int i=0;i<5;i++)
{
    break;
}

System.out.println("Hello");
```

Prints

```
Hello
```

```
public void test()
{
    return;

    // method ends here
}
```

---

## Hard Interview Questions

**Can break be replaced with a boolean flag?**

Yes.

But break is cleaner and more readable.

---

**Does break affect exception handling?**

No.

It only changes loop execution.

---

### Can break be used inside try-catch?

Yes.

---

### Is labeled break considered good practice?

Generally **avoid it** because it reduces readability.

Use it only when it clearly simplifies nested-loop logic.

---

## Top Interview Questions

1. What does break do?
  2. Which loop does break terminate?
  3. Can break exit multiple loops?
  4. What is labeled break?
  5. Difference between break and return?
  6. Can break be used outside loops?
  7. Does break terminate an if statement?
  8. Does break terminate the program?
- 
- 

## 5. continue Statement – Skip Current Iteration Intentionally

---

### Interview Rule

continue skips **only the current iteration** and moves directly to the next iteration.

Unlike break, it **does not terminate the loop**.

---

## Syntax

```
continue;
```

---

## Real-World Uses

- Skip invalid input
- Ignore negative numbers
- Skip holidays while processing attendance
- Ignore blank lines in files

Example

```
for(int i=1;i<=5;i++)
{
    if(i==3)
        continue;

    System.out.print(i+" ");
}
```

Output

```
1 2 4 5
```

---

## Hidden Interview Facts

**Q1. Does continue terminate the loop?**

No.

It skips only one iteration.

---

**Q2. What happens after continue?**

In a for loop:



```
continue
```

↓

```
Update
```

↓

```
Condition
```

↓

```
Next iteration
```

Many candidates incorrectly think it jumps directly to the condition.

---

### Q3. In a while loop?

```
while(condition)
{
    continue;
}
```

Danger!

If the update statement is below continue, it never executes.

Infinite loop.

---

Example

```
int i=0;

while(i<5)
{
    if(i==2)
        continue;
}
```

```
    i++;  
}
```

Infinite Loop.

Because i never increments after becoming 2.

Correct

```
int i=0;  
  
while(i<5)  
{  
    i++;  
  
    if(i==2)  
        continue;  
  
    System.out.println(i);  
}
```

---

## continue vs break

| continue               | break                   |
|------------------------|-------------------------|
| Skip current iteration | Stop entire loop        |
| Loop continues         | Loop terminates         |
| Used to ignore data    | Used to stop processing |

---

## Tricky Output Questions

### Question 1

```
for(int i=1;i<=5;i++)  
{
```

```
        if(i==3)
            continue;

        System.out.print(i+" ");
    }
```

Output

```
1 2 4 5
```

---

## Question 2

```
for(int i=1;i<=5;i++)
{
    if(i%2==0)
        continue;

    System.out.print(i+" ");
}
```

Output

```
1 3 5
```

---

## Question 3

```
for(int i=0;i<5;i++)
{
    System.out.print(i+" ");

    continue;

    //System.out.println("Java");
}
```

Output

```
0 1 2 3 4
```

---

#### Question 4

```
int i=0;

while(i<5)
{
    i++;

    if(i==3)
        continue;

    System.out.print(i+" ");
}
```

Output

```
1 2 4 5
```

---

## Hard Interview Questions

### Can continue be used in a switch?

Not by itself.

It can only be used if the switch is inside a loop, where it applies to the loop.

---

### Does continue skip the update expression in a for loop?

No.

The update expression still executes.

Example

```
for(int i=1;i<=5;i++)
{
    if(i==2)
        continue;

    System.out.print(i+" ");
}
```

i++ still runs after continue.

---

### Which is more dangerous inside a while loop?

continue

Because it can accidentally skip the update statement, causing an infinite loop.

---

### Is continue considered good practice?

Yes, when it simplifies the logic by filtering out unwanted cases early. Avoid excessive use that makes control flow difficult to follow.

---

## Top Interview Questions

1. Difference between break and continue?
  2. Does continue terminate the loop?
  3. What happens after continue in a for loop?
  4. Why can continue cause an infinite loop in while?
  5. Can continue be used outside loops?
  6. Can continue be used in a switch?
  7. Which statement is generally more dangerous in a while loop: break or continue?
  8. When should you use continue instead of break?
-

# ★ Ultimate Interview Trap (Asked Frequently)

Predict the output:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 2)  
        continue;  
  
    if (i == 4)  
        break;  
  
    System.out.print(i + " ");  
}
```

**Output:**

```
1 3
```

**Reason:**

- i = 1 → printed.
- i = 2 → skipped by continue.
- i = 3 → printed.
- i = 4 → loop terminates due to break.
- i = 5 is never reached.

This type of question is very common because it tests whether you truly understand the control flow of both continue and break together.